



中山大学 软件工程学院

SUN YAT-SEN UNIVERSITY SCHOOL OF SOFTWARE ENGINEERING

SSE206 《计算机网络》

第五章 网络层：控制平面

南雨宏

nanyh@mail.sysu.edu.cn

2026-04-29

提纲

- 第 1 章 计算机网络和因特网
- 第 2 章 应用层
- 第 3 章 传输层
- 第 4 章 网络层：数据平面
- 第 5 章 网络层：控制平面
- 第 6 章 链路层与局域网
- 第 7 章 无线网络和移动网络
- 第 8 章 计算机网络中的安全

第 5 章：网络层：控制平面

5.1 导论

5.2 路由选择算法

- 链路状态(LS)路由选择算法
- 距离向量(DV)路由选择算法

5.3 因特网中自治系统内部的路由选择：OSPF

5.4 ISP之间的路由选择：BGP

5.5 SDN 控制平面

5.6 ICMP：因特网控制报文协议

5.7 网络管理

- SNMP
- NETCONF/YANG

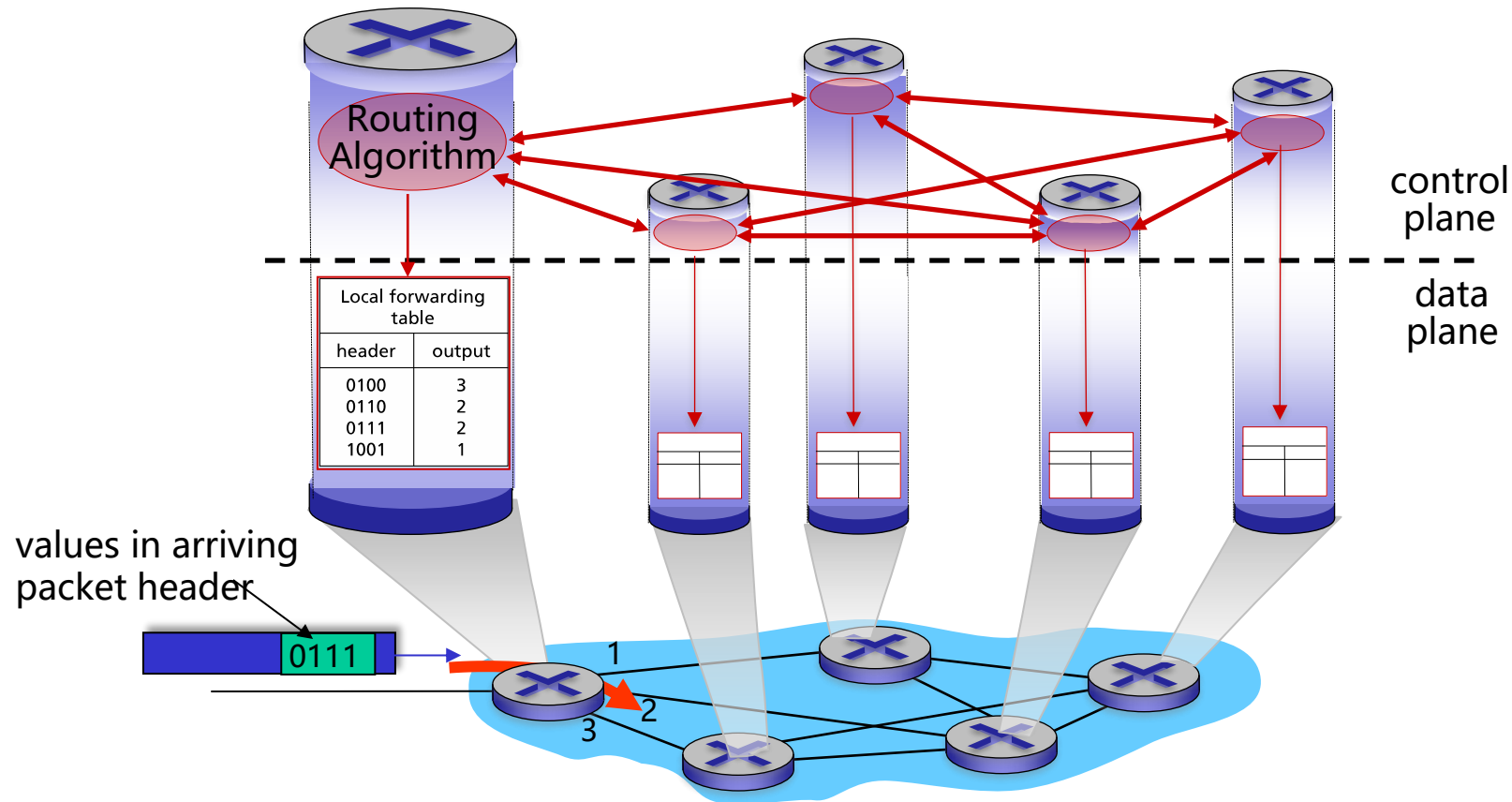
第 5 章：网络层：控制平面

本章目标:

- 理解网络层控制平面的工作原理
 - 传统路由选择算法
 - SDN 控制器
 - ICMP: Internet Control Message Protocol
 - 网络管理
- 以及它们在互联网上的实例和实现
 - OSPF, BGP, ICMP, SNMP

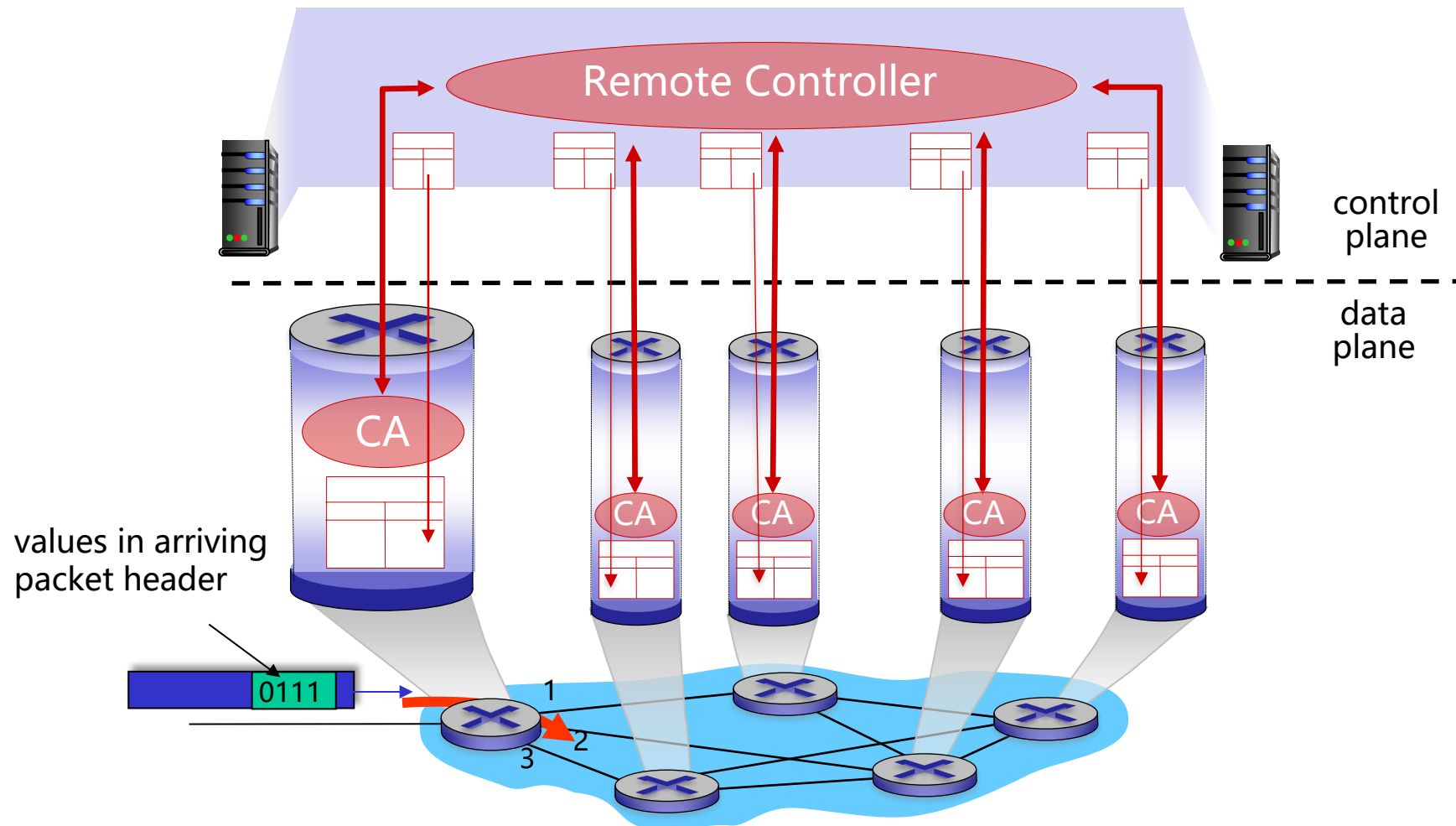
传统方式：每-路由器(Per-router)控制平面

在每一个路由器中的单独路由器算法元件，在控制平面进行交互。



SDN方式：逻辑上集中的控制平面

一个集中的（通常是远程的）控制器与本地控制代理（CAs）交互



第 5 章：网络层：数据平面

5.1 导论

5.2 路由选择算法

- 链路状态路由选择算法
- 距离向量路由选择算法

5.3 因特网中自治系统内部的路由选择

- RIP
- OSPF

5.4 ISP之间的路由选择：BGP

5.5 SDN控制平面

5.6 ICMP：因特网控制报文协议

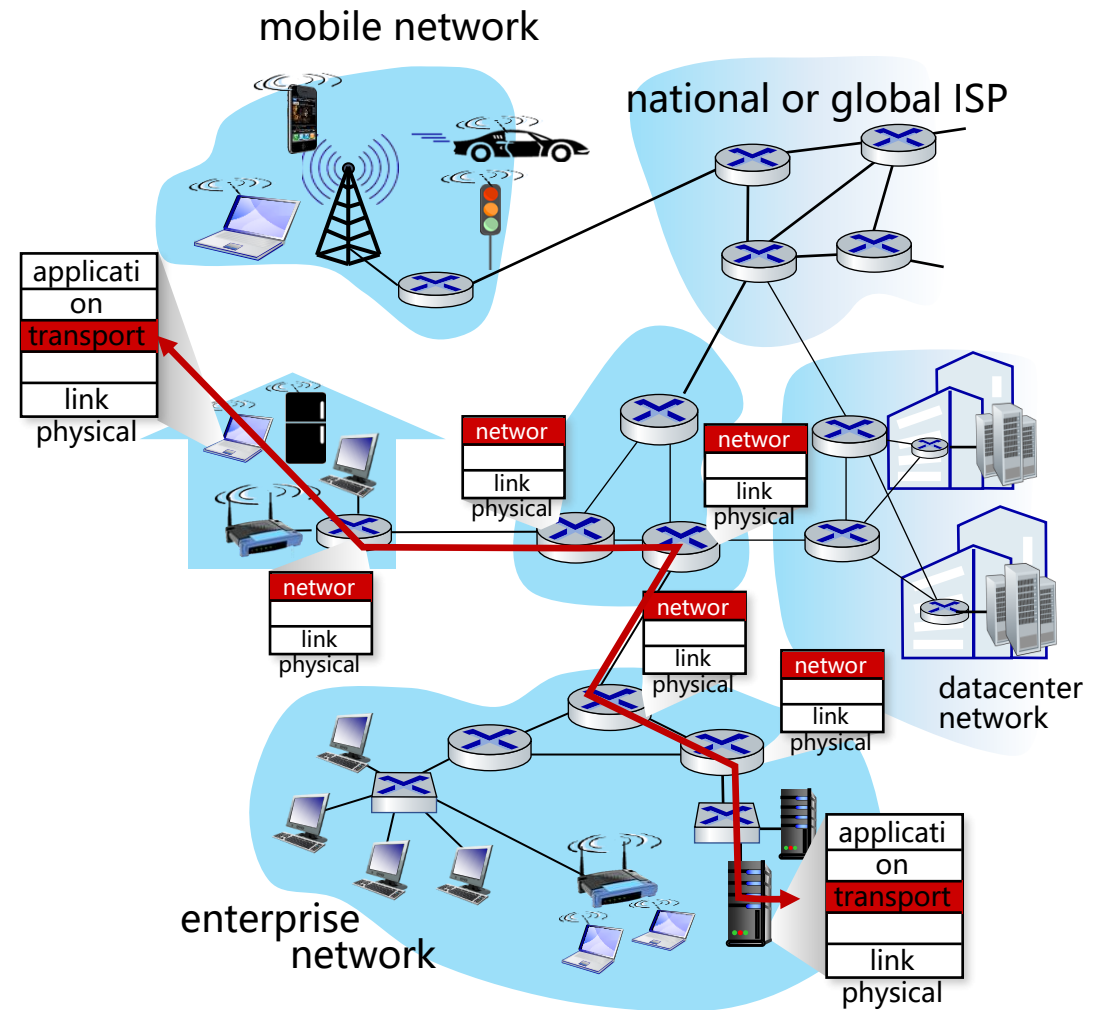
5.7 网络管理

- SNMP
- NETCONF/YANG

路由协议 (Routing protocols)

路由协议目标:

- 从发送方到接收方的过程中，确定一条通过路由器网络的好的路径（等价于路由）。
 - **Path 路径**：数据包从给定的初始源主机遍历到最终目的主机的路由器序列。
 - **"good"**：最少的“成本”，“最快的”，“最少的拥堵”。
 - routing: “十大”网络挑战之一



路由(route)的概念

- 网络-网络的路由 = 路由器 - 路由器之间路由
 - 路由器以子网到子网的路径为计算目标，而不是主机到主机
 - 路由器 - 路由器之间的最优路径 = 主机对之间的最优路径
 - 大大降低了路由计算的规模
- 在一个网络中：路由器-主机之间的通信，链路层解决（基于MAC地址）
 - 到达某个网关路由器，等同于到达了网络
- 路由选择算法(routing algorithm)：网络层软件的一部分，完成路由功能

网络的图抽象：边和路径的代价

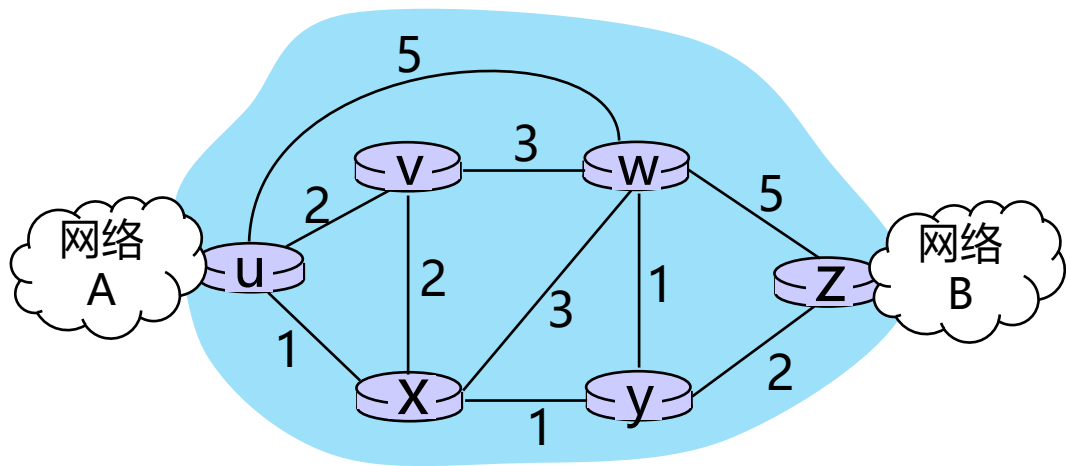


图: $G = (N, E)$

N : 路由器集合 = $\{ u, v, w, x, y, z \}$

E : 链路集合 = $\{ (u, v), (u, x), (v, x), (v, w), (x, w), (x, y), (w, y), (w, z), (y, z) \}$

路由的输入：拓扑、边的代价、源节点 Source

路由的输出：源节点的汇集树 Sink

$c_{A,B}$: 连接 A 和 B 的**直接**链路的代价

$c_{A,B}$ = 链路的代价 (A, B)

e.g., $c_{w,z} = 5$, $c_{u,z} = \infty$

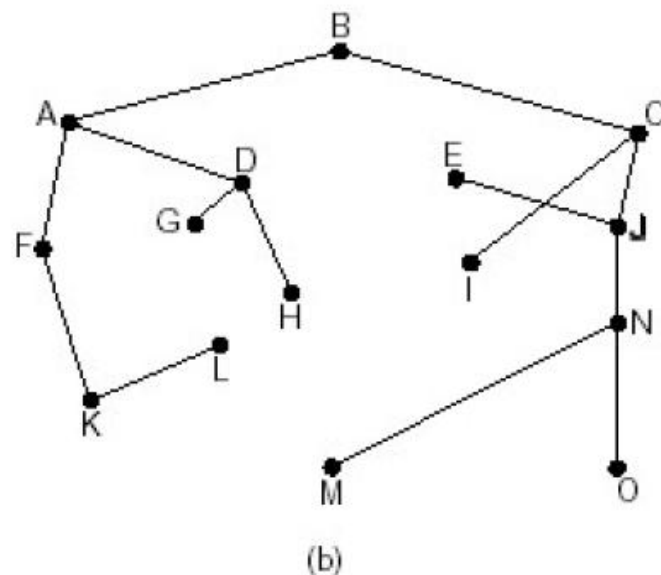
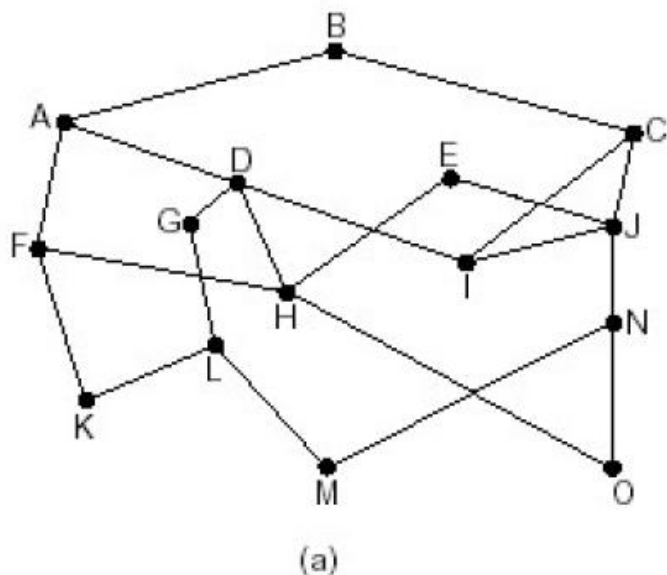
费用由网络运营商确定:

- 代价可能总为 1
- 或是链路带宽的倒数
- 或是拥塞情况

最优化原则 (optimality principle)

- 汇集树 (sink tree)

- 此节点到所有其它节点的最优路径形成的树
- 路由选择算法就是为所有路由器**找到**并**使用**汇集树



路由选择算法的原则

- 正确性 (correctness):

- 算法必须是**正确**的和**完整**的，使分组一站一站接力，正确发向目标站；
- 完整：目标所有的站地址，在路由表中都能找到相应的表项；没有处理不了的目标站地址；

- 简单性 (simplicity):

- 算法在计算机上应简单：最优但复杂的算法，时间上延迟很大，不实用，
- 不应为了获取路由信息增加很多的通信量；

- 健壮性 (robustness):

- 算法应能适应**通信量**和**网络拓扑**的变化：通信量变化，网络拓扑的变化算法能很快适应；
- 不向很拥挤的链路发数据，不向断了的链路发送数据；

路由选择算法的原则（续）

- 稳定性 (stability):
 - 产生的路由不应该摇摆
- 公平性 (fairness):
 - 对每一个站点都公平
- 最优性 (optimality):
 - 某一个指标的最优，时间、费用等指标，或综合指标；
 - 实际上，获取最优的结果代价较高，可以是次优的

路由算法分类

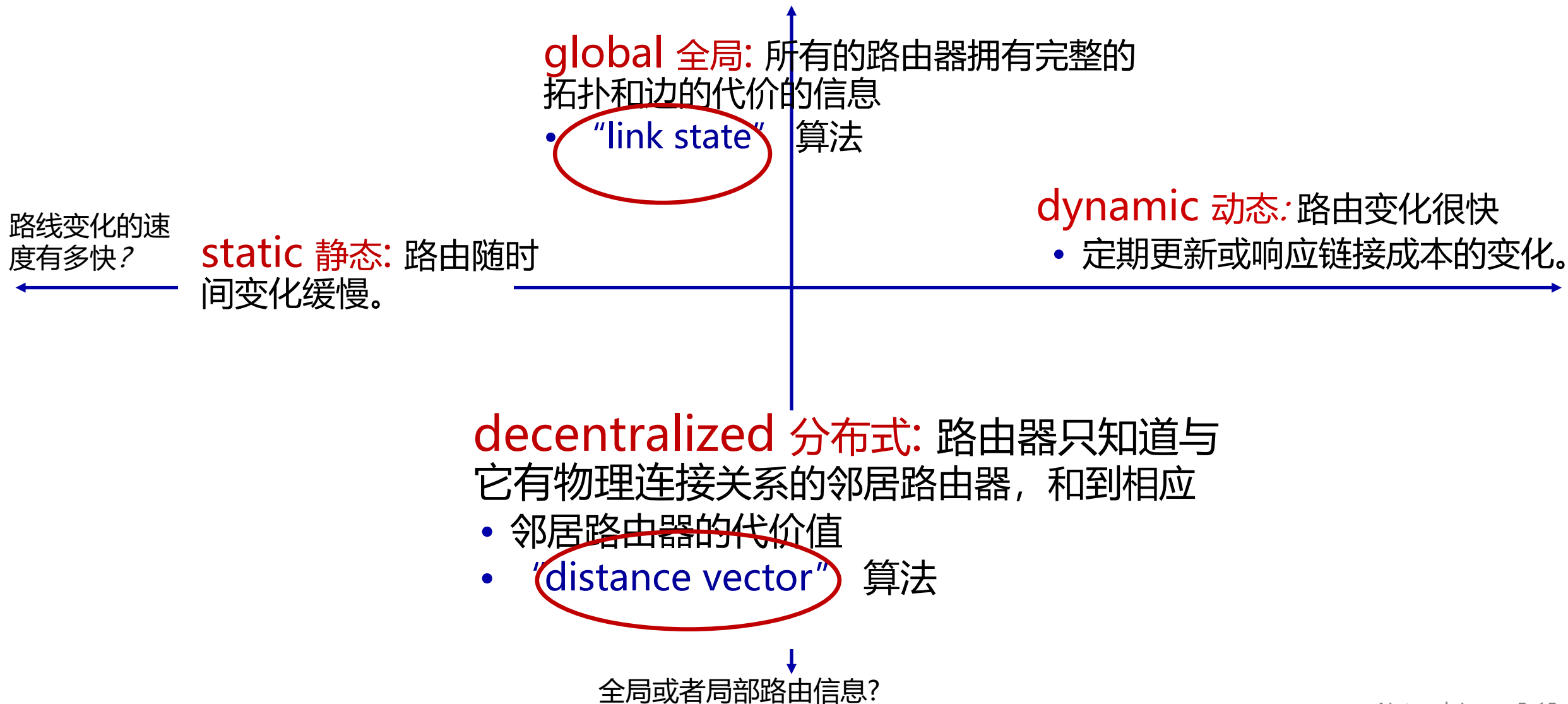
全局/局部路由信息?

- 全局:
 - 所有的路由器拥有完整的拓扑和边的代价的信息
 - “link state” LS链路状态算法
- 分布式:
 - 路由器只知道与它有物理连接关系的邻居路由器，和到相应邻居路由器的代价值
 - 叠代地与邻居交换路由信息、计算路由信息
 - “distance vector” DV距离向量算法

静态/动态?

- 静态:
 - 路由随时间变化缓慢
 - 非自适应算法(non-adaptive algorithm):
 - 不能适应网络拓扑和通信量的变化
 - 路由表是事先计算好
- 动态:
 - 路由变化很快
 - 周期性更新
 - 根据链路代价的变化而变化
 - 自适应路由选择(adaptive algorithm):
 - 能适应网络拓扑和通信量的变化

路由算法分类



第 5 章：网络层：数据平面

5.1 导论

5.2 路由选择算法

- 链路状态路由选择算法
- 距离向量路由选择算法

5.3 因特网中自治系统内部的路由选择 OSPF

5.4 ISP之间的路由选择：BGP

5.5 SDN控制平面

5.6 ICMP：因特网控制报文协议

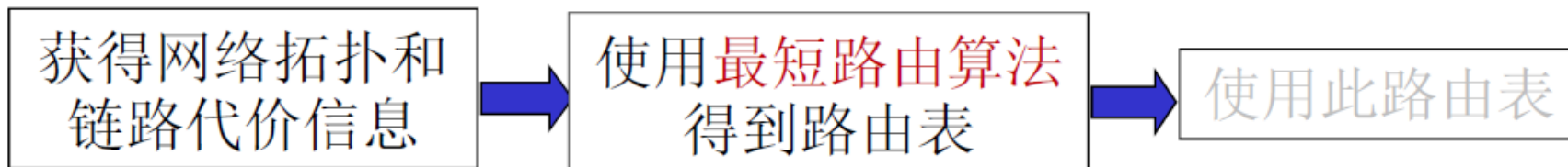
5.7 网络管理

- SNMP
- NETCONF/YANG

链路状态LS路由选择算法

■ 配置LS路由选择算法的路由工作过程

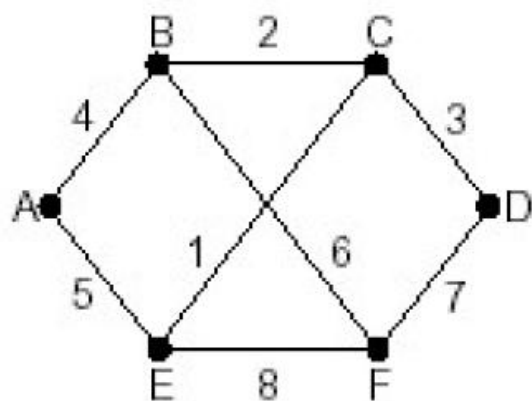
- 各点通过各种渠道获得整个网络拓扑、网络中所有链路代价等信息（该部分与算法无关，属于协议和实现）
- 使用LS路由算法，计算本站点到其它站点的最优路径（汇集树），得到路由表
- 按照路由表转发分组（datagram方式）
 - 严格意义上说不是路由的一个步骤
 - 分发到输出端口的网络层



链路状态路由选择 (link state routing)

■ LS 路由的基本工作过程

1. 发现相邻节点，获知对方网络地址
 2. 测量到相邻节点的代价（延迟、开销）
 3. 组装一个链路状态分组，描述它到相邻节点的代价情况
 4. 将分组通过扩散（泛洪）的方法发到所有其它路由器
- 通过以上4步，让**每个**路由器获得**拓扑**和**边代价**



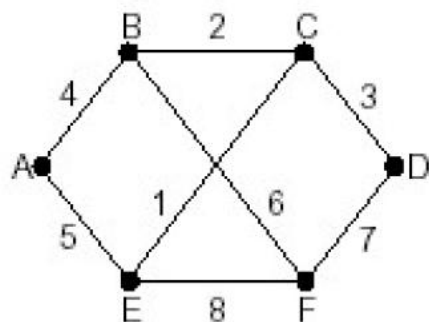
(a)

Link		State		Packets	
A	B	C	D	E	F
Seq.	Seq.	Seq.	Seq.	Seq.	Seq.
Age	Age	Age	Age	Age	Age
B 4	A 4	B 2	C 3	A 5	B 6
E 5	C 2	D 3	F 7	C 1	D 7
	F 6	E 1		F 8	E 8

(b)

链路状态路由选择 (link state routing)

- 泛洪会产生广播风暴问题，如环路
- 解决办法：
 - Age 字段：类似 TTL
 - Seq 版本号：每个路由器记录是否发送过该分组
- 泛洪是可靠的泛洪，一定要收到确认，否则一直发送
 - 确保所有节点都能够收到其他节点的链路状态



(a)

Link		State		Packets	
A	B	C	D	E	F
Seq.	Seq.	Seq.	Seq.	Seq.	Seq.
Age	Age	Age	Age	Age	Age
B 4	A 4	B 2	C 3	A 5	B 6
E 5	C 2	D 3	F 7	C 1	D 7
	F 6	E 1		F 8	E 8

(b)

链路状态路由选择 (link state routing)

■ LS 路由的基本工作过程

1. 发现相邻节点，获知对方网络地址
2. 测量到相邻节点的代价（延迟、开销）
3. 组装一个LS分组，描述它到相邻节点的代价情况
4. 将分组通过扩散（泛洪）的方法发到所有其它路由器
5. 通过Dijkstra算法找出最短路径
具体路由算法（见下页）

1-4步是让每个路由器获得
拓扑和边代价

■ LS 的应用情况

- OSPF协议：一种LS协议，被用于Internet上
- IS-IS (intermediate system- intermediate system): 被用于 Internet 主干中, Netware

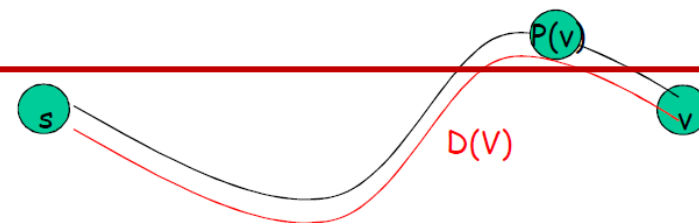
Dijkstra 算法

- **集中式**: 网络拓扑和所有的链路开销都是已知的。
 - 通过“链路状态广播”实现
 - 所有节点都有相同的信息
- 计算从一个节点(“源”)到所有其他节点的最低开销路径
 - 给出该节点的**转发表**
- **迭代**: 第k步能够知道本节点到k个其他节点的最短路径

notation

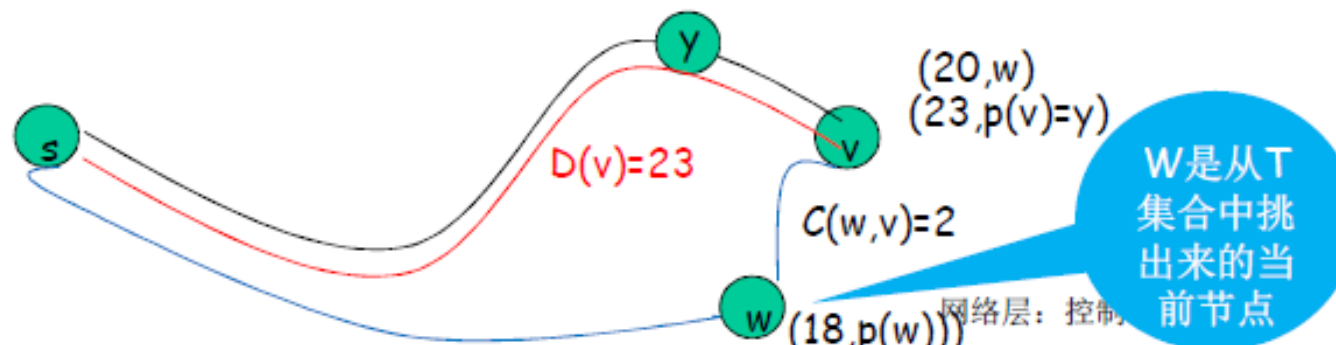
- c_{ij} : 从节点 i 到 j 链路代价 (初始状态下非相邻节点之间的链路代价为 ∞)
- $D(v)$: 从源节点到节点 v 的当前路径代价 (节点的代价)
- $p(v)$: 从源到节点 v 的路径前序节点
- N' : 当前已经知道最优路径的节点集合 (永久节点的集合)

要计算S
到其他所有节点的路由



链路状态路由选择

- 初始化
 - 除了源节点外，所有节点都为临时节点
 - 节点代价除了与源节点代价相邻的节点外，都为 ∞
- 从所有临时节点中找到一个节点代价最小的临时节点，将之变成永久节点（当前节点） w
- 对此节点的所有在临时节点集合中的邻节点（ v ）
 - 如果 $D(v) > D(w) + c(w,v)$ ，则重新标注此点， $(D(w) + C(w,v), w)$
 - 否则，不重新标注
- 开始一个新的循环



Dijkstra 算法

1 *Initialization:*

2 $N' = \{u\}$

/*计算从 u 到所有其他节点的最低开销路径*/

3 for all nodes v

4 if v adjacent to u

/* u 最初只知道直接邻居的直接路径开销*/

5 then $D(v) = c_{u,v}$

/*但可能不是最低开销!

*/

6 else $D(v) = \infty$

7

8 *Loop*

9 求 w 不在 N' 中, 使 $D(w)$ 为最小值

10 add w to N'

11 对于所有与 w 相邻且不在 N' 中的 v , 更新 $D(v)$

12 **$D(v) = \min (D(v), D(w) + c_{w,v})$**

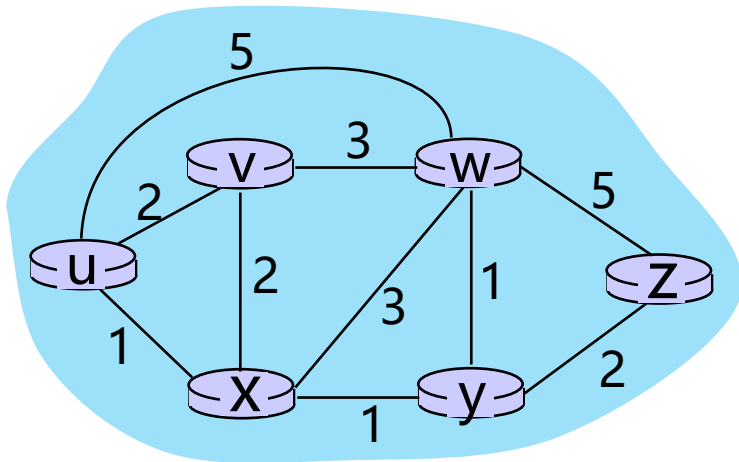
13 /* 新的到达 v 的最低开销路径, 要么是到达 v 的旧的最低开销路径

14 要么是已知的到达 w 的最低开销路径加上从 w 到 v 的直接开销*/

15 *until all nodes in N'*

Dijkstra 算法：例子

Step	N'	^v D(v),p(v)	^w D(w),p(w)	^x D(x),p(x)	^y D(y),p(y)	^z D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1						
2						
3						
4						
5						

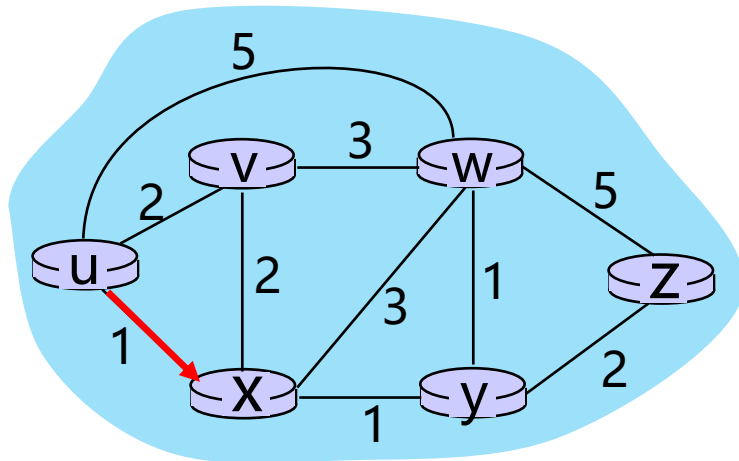


Initialization (step 0):

For all a : if a adjacent to u then $D(a) = c_{u,a}$

Dijkstra 算法：例子

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux					
2						
3						
4						
5						



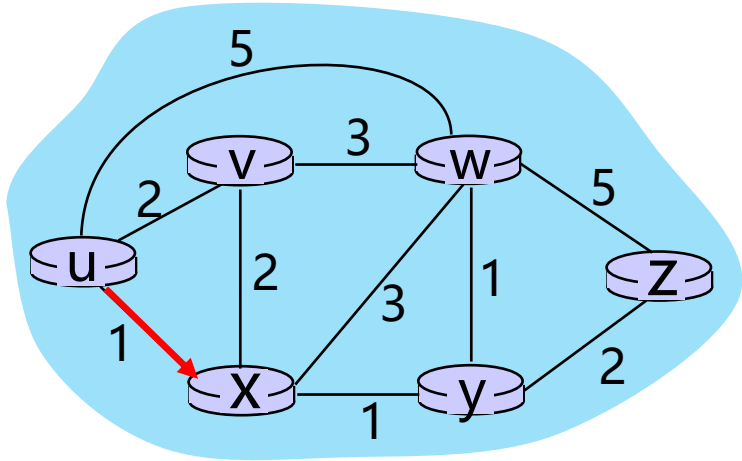
8 *Loop*

9 find a not in N' such that $D(a)$ is a minimum

10 add a to N'

Dijkstra 算法：例子

		v	w	x	y	z
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	<u>1,u</u>	∞	∞
1	ux	2,u	4,x		2,x	∞
2						
3						
4						
5						



8 *Loop*

9
10
11

find a not in N' such that $D(a)$ is a minimum
add a to N'

update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

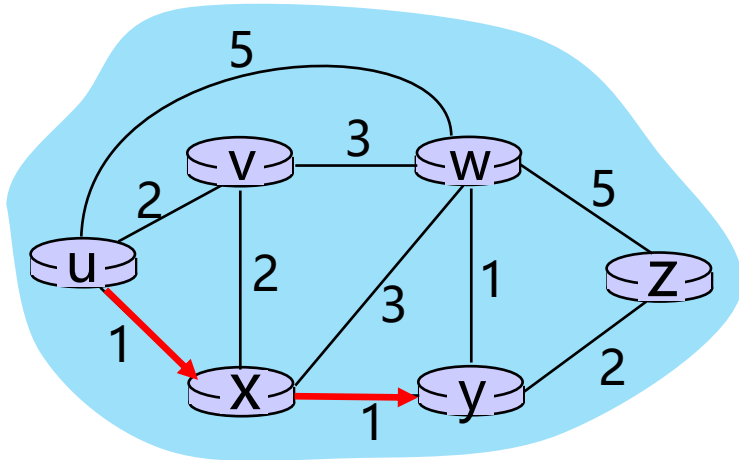
$$D(v) = \min (D(v), D(x) + c_{x,v}) = \min (2, 1+2) = 2$$

$$D(w) = \min (D(w), D(x) + c_{x,w}) = \min (5, 1+3) = 4$$

$$D(y) = \min (D(y), D(x) + c_{x,y}) = \min (\infty, 1+1) = 2$$

Dijkstra 算法：例子

		V	W	X	Y	Z
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy					
3						
4						
5						



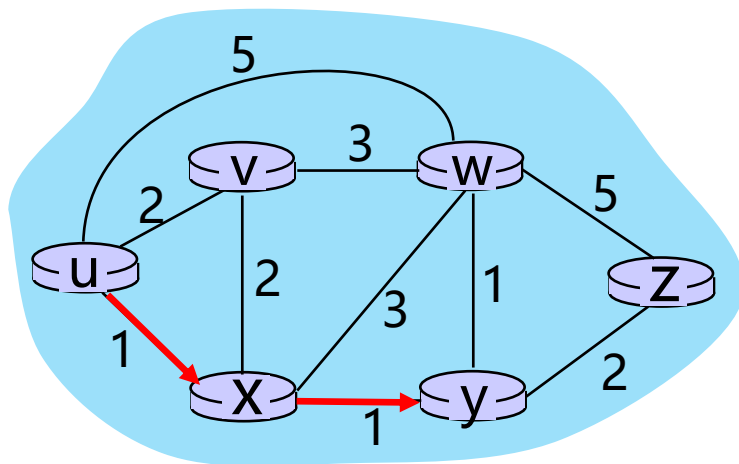
8 *Loop*

9 find a not in N' such that $D(a)$ is a minimum

10 add a to N'

Dijkstra 算法：例子

		v	w	x	y	z
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3						
4						
5						



8 *Loop*

9 find a not in N' such that $D(a)$ is a minimum

10 add a to N'

11 update $D(b)$ for all b adjacent to a and not in N' :

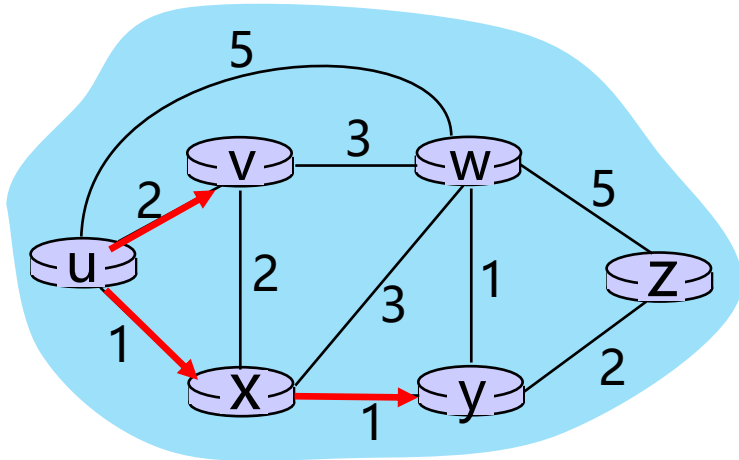
$$D(b) = \min (D(b), D(a) + c_{a,b})$$

$$D(w) = \min (D(w), D(y) + c_{y,w}) = \min (4, 2+1)$$

$$D(z) = \min (D(z), D(y) + c_{y,z}) = \min (\infty, 2+2) = 4$$

Dijkstra 算法：例子

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x	2, x	∞	∞
2	uxy	2, u	3, y		4, y	
3	uxyv					
4						
5						



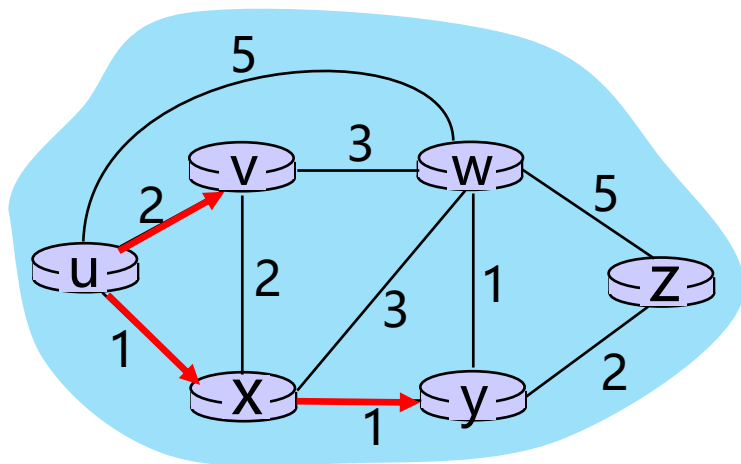
8 *Loop*

9 find a not in N' such that $D(a)$ is a minimum

10 add a to N'

Dijkstra 算法：例子

		v	w	x	y	z
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x	2,x	∞	∞
2	uxy	2,u	3,y		4,y	
3	uxyv		3,y		4,y	
4						
5						



8 Loop

9 find a not in N' such that $D(a)$ is a minimum
10 add a to N'

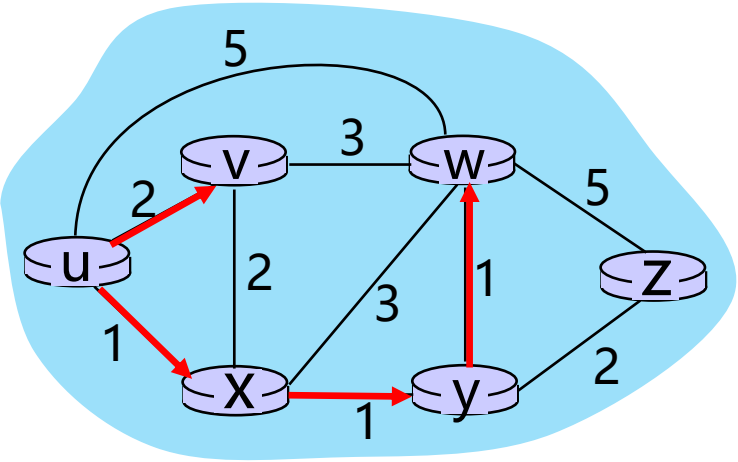
11 update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

$$D(w) = \min (D(w), D(v) + c_{v,w}) = \min (3, 2+3) = 3$$

Dijkstra 算法：例子

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x	2, x	∞	∞
2	uxy	2, u	3, y	4, y	∞	∞
3	uxyv	2, u	3, y	4, y	∞	∞
4	uxyvw	2, u	3, y	4, y	∞	∞
5						



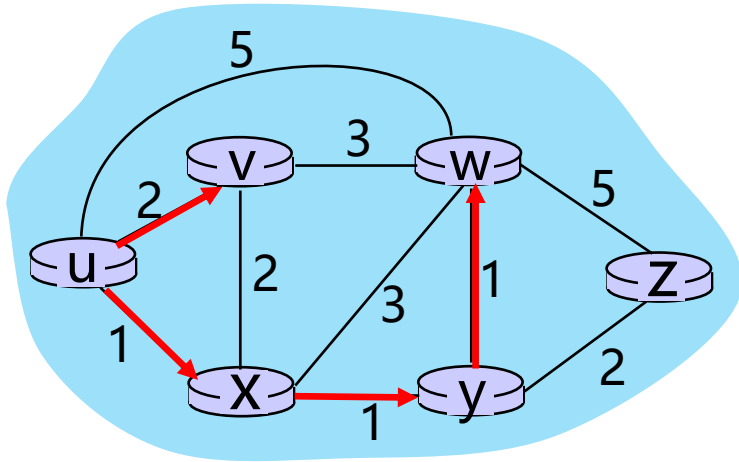
8 *Loop*

9 find a not in N' such that $D(a)$ is a minimum

10 add a to N'

Dijkstra 算法：例子

		v	w	x	y	z
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	<u>1,u</u>	∞	∞
1	ux	2,u	4,x		<u>2,x</u>	∞
2	uxy	<u>2,u</u>	3,y			4,y
3	uxyv		<u>3,y</u>			4,y
4	uxyvw					4,y
5						



8 *Loop*

9 find a not in N' such that $D(a)$ is a minimum
10 add a to N'

11

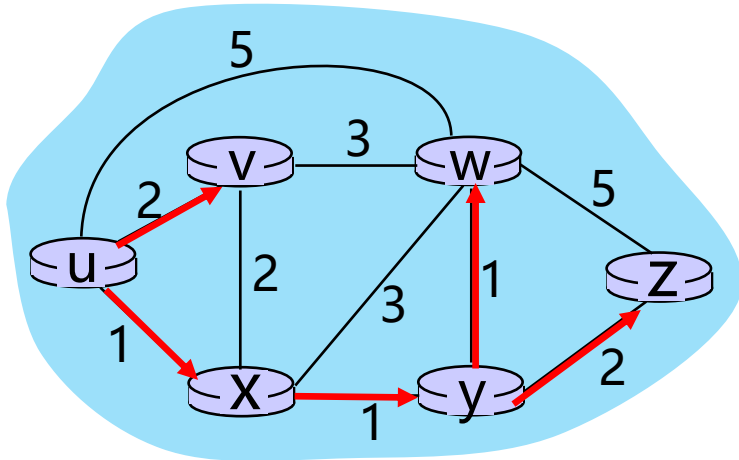
update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

$$D(z) = \min (D(z), D(w) + c_{w,z}) = \min (4, 3+5) = 4$$

Dijkstra 算法：例子

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					4, y
5	uxyvwz					



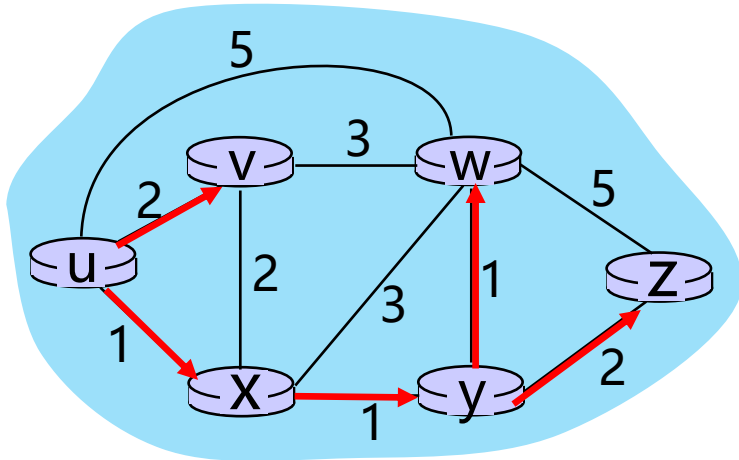
8 *Loop*

9 find a not in N' such that $D(a)$ is a minimum

10 add a to N'

Dijkstra 算法：例子

		v	w	x	y	z
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					



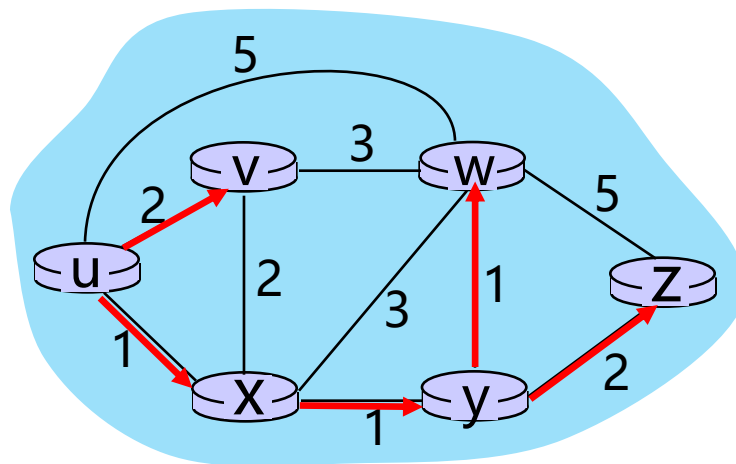
8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'
- 11 update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

Dijkstra 算法：例子

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					4, y
5	uxyvwz					



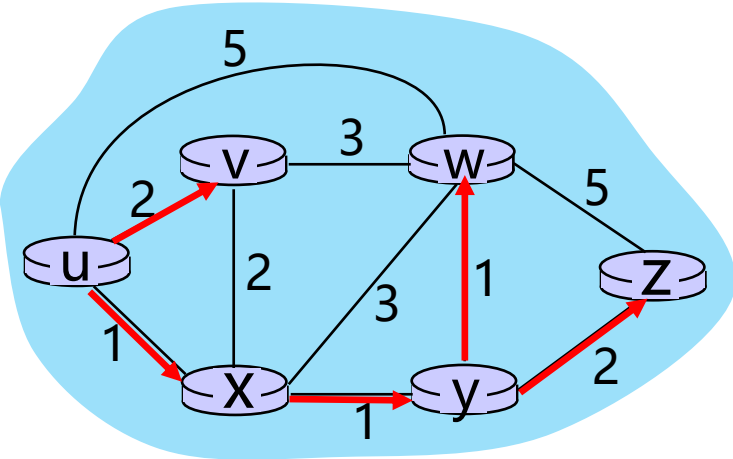
Initialization (step 0): For all a : if a adjacent to then $D(a) = c_{u,a}$

find a not in N' such that $D(a)$ is a minimum
add a to N'

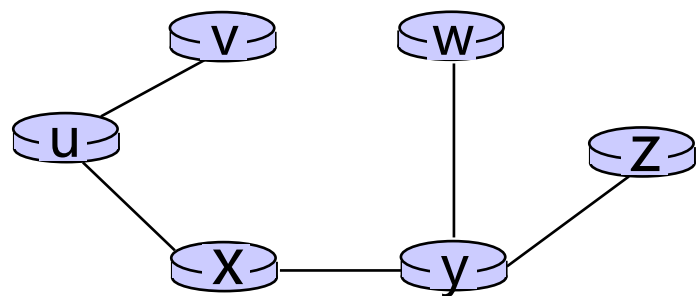
update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

Dijkstra 算法：例子



由 u 得到的最低开销路径树:



u 中得到的转发表:

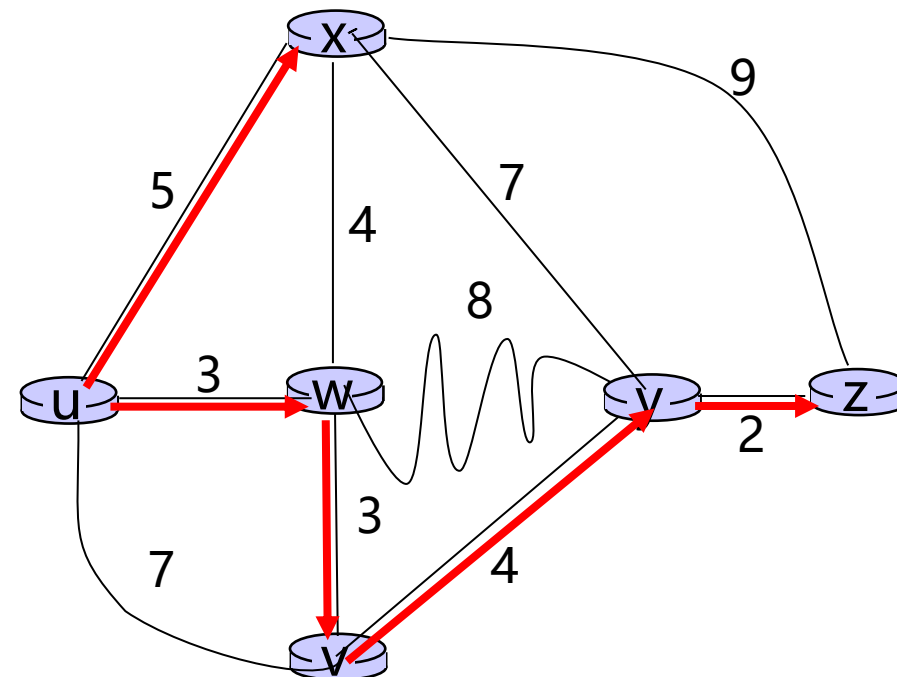
destination	outgoing link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

直接从 u 到 v 的路由

从 u 经过 x 到所有其他目的地的路线

Dijkstra 算法：另一个例子

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	7, u	3, u	5, u	∞	∞
1	uw	6, w		5, u	11, w	∞
2	uwvx	6, w			11, w	14, x
3	uwvxv				10, v	14, x
4	uwxvy					12, y
5	uwxvyz					



notes:

- 通过跟踪前一个节点来构造最低开销路径树
- 联系可以存在（可以任意打破）

Dijkstra 算法的讨论

- **算法复杂度 (algorithm complexity) :** n 个节点
 - 每一次迭代: 需要检查所有不在永久集合 N 中节点
 - $n(n+1)/2$ 次比较: $O(n^2)$
 - 有更加高效的实现: $O(n\log n)$
- **消息复杂度 (message complexity) :**
 - 每个路由器必须**广播**它的链路状态信息给其他 n 个路由器
 - 每个路由器的消息经过 $O(n)$ 条链路: 总体消息复杂度: $O(n^2)$